STUDIO BIBLE

HOW TO BE HUMAN

Full Production Plan & Specification

DRUNKEN DONUTS
Mediadesign Hochschule · July 2026

PLAN.md — How To Be Human: Full Production Rebuild

Status: phase 0 (bootstrap) in progress. Update the phase table at the bottom as phases land. The behavioral spec for gameplay is the prototype repo at `../HowToBeHuman/ClaudePrototype/HowToBeHuman` — the old repo is the *spec*, not the starting point. The detailed requirements live in `SPEC.md`.

1 Vision

Rebuild How To Be Human from scratch as three cleanly separated parts: a pseudo-engine (carries exactly this game's workload, nothing more), the game built on it, and a central editor (PySide6) that is the designer's only interface — balancing, map design, asset importing, and Claude agent dispatch all happen in the editor.

Design pillars (tie-breakers for every future decision):

1. Agent legibility — small single-purpose files, one canonical data format with schemas, no editor-only hidden state. The repo must stay cheap for Claude agents to read and safe for them to edit. (This is *why* the project stays Python instead of moving to Godot/Unity.)
2. Strict layering — game logic never touches pygame; editor and game never import each other; both consume `engine/` and `data/`.
3. Editor is the designer interface — humans never hand-edit JSON. JSON persists underneath (diffable by git, writable by agents) but it is an implementation detail, like Unity's `.meta` files.

2 Repo bootstrap (first commit)

- `.gitignore`: `build/`, `dist/`, `__pycache__/`, `.venv/`, `*.exe`, editor prefs, logs. Built artifacts are never committed (fixes the prototype repo's dirty-binary-files problem). Committed: source, `data/` JSONs, source sprite sheets in `data/sprites/`.
- Folder skeleton with a CLAUDE.md router at root + one CLAUDE.md per package.
- `PLAN.md` (this file), `SPEC.md`, `requirements.txt` (`pygame-ce`, `PySide6`, `Pillow`, `jsonschema`), `README.md`.

```
HowtobeHumanFullProd/
├─ CLAUDE.md                # router → one package doc per task
├─ PLAN.md  SPEC.md  .gitignore  requirements.txt  README.md
├─ .claude/                 # commands (start-domain etc.), hooks, locks integration
├─ engine/                  # the pseudo-engine
│  ├─ CLAUDE.md
│  ├─ core/                 # loop, GameObject, Component, Transform, Scene, events
│  ├─ coords/               # THE coordinate system (world⇔screen, iso projection)
│  ├─ render/               # RenderItem collection, depth sort, pygame backend
│  ├─ physics/              # movement, range/radius queries, spatial grid
│  └─ assets/               # manifest, slot registry, animation slicing, placeholder
├─ game/
│  ├─ CLAUDE.md
│  ├─ main.py               # window host: engine → pygame window
│  └─ buildings/ enemies/ map/ ui/ core/
├─ editor/                  # PySide6 application
│  ├─ CLAUDE.md
│  ├─ main.py               # shell, docking layout
│  ├─ panels/               # selector, viewport, balancing, asset import
│  ├─ run_controls.py       # Play / Build / Playbuild
│  ├─ spawnclaude.py        # launch scoped Claude sessions
│  └─ locks.py
├─ data/                    # single source of truth (editor + agents write; game reads)
│  ├─ CLAUDE.md
│  ├─ schemas/              # JSON Schema per file type — all writers validate
│  ├─ balancing/            # per-domain balancing JSON (locks live here)
│  ├─ maps/                 # map files + active_map selector
│  └─ sprites/              # imported sheets + asset_manifest.json
└─ tools/                   # headless smoke test, build script (PyInstaller)
```

3 Engine

Coordinate system (`engine/coords/`) — the linchpin, built first. One authority for world space (fractional tile coords) → screen pixels (iso projection + camera offset + zoom) and the exact inverse (screen → world, needed for click-to-paint in the map editor and tile picking in game). Nothing outside this module converts coordinates. Geometry constants (tile pitch, map dims) are data-driven from `data/`.

GameObject model (`engine/core/`) — hybrid, with the sanity rule: components are what the editor sees; subclasses are behavior convenience.

- `GameObject`: id, name, `Transform` (world pos, layer/height), list of `Components`, lifecycle (spawn/update/despawn).
- `Component`: serializable data + optional `update(dt)`. Core set: `SpriteAnimator` (asset key + current animation state), `Health`, `Movement`, `RangeSensor`, plus game-defined ones. Phasing: `SpriteAnimator` + `Health` ship in phase 2; `Movement` + `RangeSensor` ship with `engine/physics` (whose queries they wrap) at the start of phase 9.
- Game code subclasses `GameObject` (e.g. `Building(GameObject)`) to wire components in `__init__` — but no gameplay state outside components, so the inspector and the serializer see everything generically.
- `Scene`: owns objects, drives update order, queryable (by type/tag/area).

Render pipeline (`engine/render/`) — three strict layers:

1. Game objects submit visual data each frame — `RenderItem(asset_key, animation, frame_time, world_pos, layer, tint, …)`. No pixels, no pygame.
2. The renderer layer collects all RenderItems, resolves each to a concrete surface+frame via the asset system, sorts for iso depth, converts every world position to pixels via `engine/coords`, producing a flat

draw list.

3. The pygame backend blits the draw list to a target `Surface` — the *game window* in game, an *offscreen surface shown in a Qt widget* in the editor. Same pipeline, two hosts.

Game logic is fully headless-testable (layers 2–3 simply not invoked).

Physics (`engine/physics/`) — deliberately simple: waypoint/path movement, range & radius queries (spatial grid so towers don't scan all enemies), tile occupancy. No forces, no collision response.

Asset system (`engine/assets/`) — the prototype's asset-importer model, generalized to *all* game visuals:

- Universal slot registry: every renderable thing declares an asset slot — buildings (type/tier/level), enemies, tiles, UI elements, VFX. Slot = key + frame size + category. Registry is data-driven, not hardcoded per family.
- Manifest (v2 of the prototype format): per entry — sheet path, frame_w/h, offset_x/y, rows[] with `{animation, fps, hidden[], loop_start/end/count}`. Row 0 = idle. Same `playback_order` semantics (pre-roll, looped range × count, post-roll, hidden frames dropped).
- Placeholder: any slot without a manifest entry renders a default grey X (at the slot's frame size) — in game *and* editor. No procedural art system in the new engine; grey X is the universal "no asset yet" state.
- Animation vocabulary extensible per category (buildings: idle/attack/death/hurt/place/upgrade; enemies: idle/walk/attack/death; UI/VFX as needed).

4 Data (`data/`)

- JSON is the only value store — the prototype's py+json dual system is dead. Python is loader code only. Every file type has a JSON Schema in `data/schemas/`; editor and agents validate on write; game validates on load (fail loud in dev).
- `data/balancing/` — per-domain balancing files, carrying the `_lock` field for the branch+lock protocol.
- `data/maps/` — any number of map files (tile grid, zone rings, spawn points, base position) + an `active_map` pointer the game reads; the editor has a selector for which map is live.
- `data/sprites/` — imported sheets + `asset_manifest.json`.

5 Editor (PySide6)

One window, docked panels, everything selection-driven:

- Selector panel (tree): Map(s), Buildings (type→tier→level), Enemies, UI, VFX. Selecting a node drives the other panels. Assigned-asset markers (□).
- Viewport panel (embedded engine render surface):
  - Map selected → tilemap editor (Godot-style): tile palette + paint / erase / line / rect / bucket / picker tools; layers with visibility toggles (terrain, zone tint, base, deco-above-entities); spawning is a painted zone, base is the single draggable map object; right-drag pan, cursor-centered zoom, ghost preview; global Ctrl+Z/Y undo; create/duplicate/save maps; set active map. No autotiling.
  - Entity selected → entity preview: renders it via the real engine pipeline in idle animation, dropdown to preview any authored animation, range-radius overlay. Grey X if no asset.

- Balancing panel (below viewport): auto-generated form from the selected object's schema — spinboxes/sliders/dropdowns with validation; writes to `data/balancing/`. Respects domain locks (locked → read-only + owner shown).
- Asset import (contextual on selection): full parity with the prototype importer — import sheet PNG, per-row animation/fps/hidden/loop editors, offset nudge, animated preview, clear-to-placeholder — writing manifest v2.
- Run controls: Play = subprocess `py game/main.py` · Build = run PyInstaller · Playbuild = launch built exe. Editor stays open; data is saved before launch.
- Spawnclaude: pick a domain (or "small tweak / no lock") → editor sets the lock, opens a terminal running `claude` in the repo with the domain pre-locked. Locked domains greyed out. `/merge-domain` remains the only unlock.
- Access limitations: lock display/enforcement in-editor now; further ideas slot in later without structural change (the lock system is the single enforcement point).

6 Build order

Each phase ends runnable. Per the project's TDD workflow: every phase starts with an acceptance checklist and a runnable test, then implementation iterates until green.

Phase	Deliverable	Proof it works	Status
0	Repo bootstrap: gitignore, skeleton, CLAUDE.mds, PLAN.md, SPEC.md	tree matches plan; docs in place	in progress
1	`engine/coords` + render pipeline + asset placeholder	headless test: world⇔screen round-trips; grey-X grid renders to offscreen surface	done — `py -m unittest discover -s tools/tests -t .` (28 tests); visual check `py tools/render_demo.py`
2	GameObject/Component/Scene (`SpriteAnimator` + `Health` only) + `game/main.py` window host + minimal `tools/smoke.py` (T-2)	scrolling iso map of grey-X tiles in a pygame window; smoke test prints OK	done — `py -m unittest discover -s tools/tests -t .` (58 tests); `py tools/smoke.py` OK; live window ~60fps
3	Qt viewport spike — engine surface inside PySide6 at 60fps	same grid inside the editor window *(riskiest integration — done early on purpose)*	—
4	Data schemas + selector panel + balancing panel	edit a value in editor → validated JSON on disk → game subprocess loads it	—
5	Asset system v2 + import panel + entity preview	import a sheet for one building, preview animations in editor, see it in game	—
6	Map format + tilemap editor + active-map selector	paint a map, save, Play launches game on it	—

Phase	Deliverable	Proof it works	Status
7	Run controls (Play/Build/Playbuild)	all three buttons work end-to-end	—
8	Spawnclaude + locks + `.claude/` commands ported	dispatch a locked-domain agent from the editor	—
9+	`engine/physics` (E-30..E-32) + `Movement`/`RangeSensor` components first, then port gameplay domain-by-domain (map → buildings → enemies → core phases → ui), prototype repo as spec	each domain: acceptance checklist → test → implement → live playtest	—

7 Risks / open items

- Qt + pygame embed is the one integration with real unknowns → phase 3 spike, immediately after the engine can render at all. Fallback exists (render to QImage) but costs a frame copy.
- Access limitations: more ideas pending from the designer; architecture keeps locks as the single enforcement point so additions don't restructure anything.
- Manifest migration: the prototype's imported sheets + `sprite_manifest.json` convert to manifest v2 via a one-off script during phase 5 — existing art carries over.

SPEC.md — How To Be Human: Full Production Specification

Companion to `PLAN.md` (which owns the build order and phase status). This document is the requirement-level outline of *what* each part of the project must do. Requirements are numbered per section (`E-*` engine, `D-*` data, `G-*` game, `ED-*` editor, `T-*` tooling) so tasks, tests, and PRs can reference them.

The behavioral spec for gameplay is the prototype repo (`../HowToBeHuman/ClaudePrototype/HowToBeHuman`): unless this document says otherwise, "what the prototype does" is the required behavior.

1 Overview

- Product: isometric tower defence. The player spends *love* to unlock tiles and place musicians/defenders that protect "the hole" from enemy waves.
- Deliverables: the game (Windows exe via PyInstaller) and the editor (run from source; the designer's single interface to all game data).
- Stack: Python 3.11+, pygame-ce (game rendering), PySide6 (editor shell), Pillow (image slicing), jsonschema (data validation).

1.1 Design pillars

1. Agent legibility — small single-purpose files; one canonical data format with schemas; no state that only the editor can see.
2. Strict layering — game logic never touches pygame; `editor/` and `game/` never import each other; both consume `engine/` and `data/`.
3. Editor is the designer interface — humans never hand-edit JSON; agents and the editor both write it through schema validation.

1.2 Non-goals

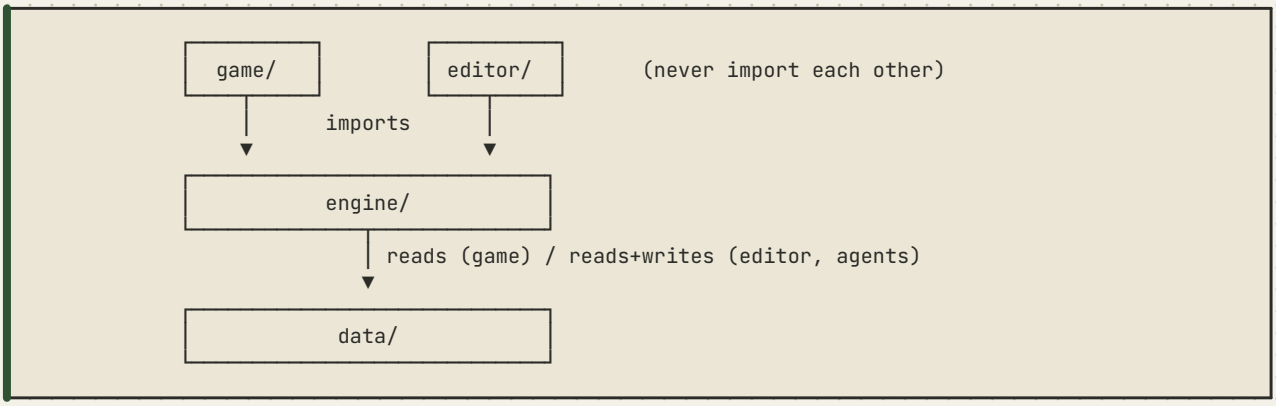
- Not a general-purpose engine: it carries exactly this game's workload.
- No forces/impulse physics, no collision response.
- No procedural sprite art (the prototype's `sprite_gen` is not ported); the grey-X placeholder is the only "no asset" render.
- No in-process play-in-editor; Play is always a subprocess.
- No multiplayer, no non-Windows packaging (for now).

2 Terminology

Term	Meaning
World space	Fractional tile coordinates `(col, row)`; the only space game logic uses.

Term	Meaning
Screen space	Pixels on the render target after iso projection + camera.
Slot	A named asset attachment point (e.g. `stone_thrower_t1_lvl1`, `tile_buildable`, `vfx_explosion`).
Manifest	`data/sprites/asset_manifest.json` — maps slots to sheets + animation metadata.
Domain	One of `buildings / enemies / map / ui / core` — the unit of balancing files, locks, and agent scoping.
Active map	The map file the game loads, chosen in the editor.

3 Architecture & layering rules



Hard rules (enforceable by review and lint):

- `engine/core`, `engine/coords`, `engine/physics`, `engine/assets` (metadata half) import no pygame. Only `engine/render`'s backend and the surface cache touch pygame.
- `game/` imports `engine/`, never `editor/`. `editor/` imports `engine/`, never `game/`.
- All coordinate conversion goes through `engine/coords`. No other module may do iso math.
- All disk writes to `data/` go through the schema-validating writer in `engine/` (used by editor and by agents' loader code).

4 Engine (`engine/`) — requirements `E-*`

4.1 Coordinate system (`engine/coords/`)

- E-1 One `CoordinateSystem` owning: tile pitch (w/h), map dimensions, camera (pan px, zoom), all loaded from data — no hardcoded geometry.
- E-2 `world_to_screen(wx, wy) -> (px, py)` for fractional world coords; applies iso projection, camera pan, zoom.
- E-3 `screen_to_world(px, py) -> (wx, wy)` — exact inverse of E-2 (round-trip error < 1e-6 at zoom 1). Basis for tile picking in game and click-to-paint in the editor.
- E-4 Depth key function for iso draw ordering (by world position + layer), used only by the renderer.

- E-5 Camera: pan, clamp to map bounds, zoom levels; pure state — input mapping lives in the host (game or editor).

4.2 Game object model (`engine/core/`)

- E-10 `GameObject`: stable id, name, `Transform` (world pos, layer), ordered component list, lifecycle hooks (`on_spawn`, `update(dt)`, `on_despawn`).
- E-11 `Component`: serializable fields (declared, typed) + optional `update(dt)`. All gameplay state lives in components — subclasses may add behavior/methods but not authoritative state fields.
- E-12 Core components shipped by the engine: `SpriteAnimator` (slot key, current animation, phase offset), `Health`, `Movement` (waypoint follower), `RangeSensor` (radius / Chebyshev range). Delivery is phased: `SpriteAnimator` + `Health` land with the core model (phase 2); `Movement` + `RangeSensor` land together with the `engine/physics` primitives they wrap (E-30/E-31), ahead of the phase-9 gameplay port.
- E-13 `Scene`: owns objects; spawn/despawn queues applied at frame boundaries; iteration by type/tag; area queries delegate to physics grid.
- E-14 Fixed-order frame: input → `Scene.update(dt)` → render submit. `dt` scaling (combat speed) is applied by the game host, not the engine.
- E-15 Serialization: any GameObject can be dumped to/loaded from a JSON dict (components with declared fields). This is what the editor inspects.

4.3 Render pipeline (`engine/render/`)

- E-20 `RenderItem`: `(slot_key, animation, anim_time_ms, world_pos, layer, tint?, flip?, overlay?)`. Game objects (via `SpriteAnimator`) emit RenderItems; they never compute pixels and never touch pygame.
- E-21 `Renderer.submit(item)` / `Renderer.flush(target_surface)`: resolve each item to a concrete frame via the asset system, depth-sort (E-4), convert positions via coords, blit to the target.
- E-22 Target-agnostic: same pipeline draws to the game window surface and the editor's offscreen viewport surface.
- E-23 Missing asset → grey-X placeholder at the slot's frame size (E-33); rendering never raises on missing art.
- E-24 Overlay pass for non-sprite draws (range circles, tile highlights, debug grid) — same coordinate authority, drawn after sprites.
- E-26 Named draw layers in fixed order: ground (tiles) → entities (buildings/enemies, iso depth-sorted) → deco (map deco sprites, always above entities — e.g. trees covering spawn tiles) → overlay (E-24) → HUD.
- E-25 Performance target: full map + 100 animated objects at 60 fps at 1080p (prototype-scale load).

4.4 Physics (`engine/physics/`)

- E-30 Waypoint path movement: follow a list of world-space waypoints at a speed; expose progress + arrival events. (Pathfinding itself is game-side, ported from the prototype's `pathfinder.py`.)
- E-31 Spatial grid: insert/move/remove objects; query by radius and by Chebyshev tile range without full scans.

- E-32 Tile occupancy: which object occupies a tile; queried by placement logic and the map editor.

4.5 Asset system (`engine/assets/`)

Generalization of the prototype's asset importer (`tools/asset_importer.py` + `src/core/sprite_manifest.py`) to all game visuals: buildings, enemies, tiles, UI, VFX, deco.

- E-33 Grey-X placeholder: generated at any requested frame size; used by game and editor identically for slot-without-entry.
- E-34 Slot registry, data-driven: slots are declared in `data/schemas`-validated data (per category: frame size, animation vocabulary, grouping for the editor tree) — not hardcoded per family as in the prototype.
- E-35 Manifest v2 (see D-30): loader slices sheets into `{slot: {animations: {name: [(frame, dur_ms)]}, offset}}` with the prototype's exact row semantics: rows = animations, row 0 = idle (required); per-row fps, hidden frames, loop `start/end × count` (`playback_order` pre-roll → looped range × count → post-roll, hidden dropped); per-entry frame size and placement offset.
- E-36 `current_frame(slot, animation, time_ms, phase_ms)` — pure function of time; falls back to idle when the requested animation has no row; returns placeholder sentinel when the slot has no entry.
- E-37 Corrupt/missing manifest or sheet: log and fall back to placeholders; never crash the game or editor at load.
- E-38 Migration tool: one-off converter from the prototype's `sprite_manifest.json` + `assets/sprites/imported/` to manifest v2 (built in phase 5).

---

5 Data (`data/`) — requirements `D-*`

5.1 Principles

- D-1 JSON files under `data/` are the only value store. No balance or content values in Python. (The prototype's py+json dual system is dead.)
- D-2 Every file type has a JSON Schema in `data/schemas/`. The editor and agent-facing writer validate on save; the game validates on load and fails loud in dev.
- D-3 Files are formatted deterministically (sorted keys, 2-space indent) so git diffs are minimal and agent edits are cheap.
- D-4 Designers never open these files; the editor is the interface. Agents may edit them directly but must validate against the schema.

5.2 Balancing (`data/balancing/`)

- D-10 One file per domain: `buildings.json`, `enemies.json`, `map.json`, `ui.json`, `core.json`.
- D-11 Each file carries a `_lock` field: `"UNLOCKED"` or `{"locked_by": <branch/agent>, "since": <iso date>}`. Lock semantics follow the branch+lock protocol (§8).
- D-12 Value semantics (×10 combat scale, BASE_HP exception, phase timers, XP tables, etc.) are carried over from the prototype's balancing modules; the schema documents units and scale per key.

5.3 Maps (`data/maps/`)

- D-20 Map file: id, display name, grid dimensions, and layers: the terrain/zone grid (buildable / combat / spawning / background variants — spawning is a painted tile zone, NOT point objects; enemies enter from spawning-zone tiles as in the prototype), the deco layer (placed deco sprites with world positions; renders ABOVE entities, E-26), and the base ("hole") position — the single movable map object.
- D-21 `active_map.json`: single pointer to the map the game loads. Set only via the editor's selector.
- D-22 Any number of maps may exist; creating/duplicating/deleting maps is an editor operation.

5.4 Asset manifest v2 (`data/sprites/`)

- D-30 `asset_manifest.json`: `{version: 2, entries: {slot_key: {sheet, frame_w, frame_h, offset_x, offset_y, rows: [{animation, frames, fps, hidden[], loop_start, loop_end, loop_count}]}}}` — field semantics identical to the prototype manifest, plus mandatory version and per-entry frame size.
- D-31 Imported sheets are copied to `data/sprites/imported/<slot>.png` on import (source PNGs are committed; they are content, not build artifacts).
- D-32 Slot declarations (which slots exist, per category) live in data — `data/schemas/slots.*` — consumed by E-34 and by the editor tree.

---

6 Game (`game/`) — requirements `G-*`

The prototype defines the behavior; this section lists the systems to port and where they land. Port order and per-domain acceptance work is PLAN.md phase 9+.

- G-1 `game/main.py`: window host — creates the pygame window, runs the engine loop, routes input. The only entry point (`py game/main.py`).
- G-2 `game/map/`: tile map load from active map file; pathfinding (ported); camera input mapping; tile picking via `engine/coords`.
- G-3 `game/buildings/`: building types as `GameObject` subclasses wiring components — defence, aoe defence, economic (musicians), meditator, boost (speed/damage/hp), painter, sun scorcher, blocker, wall builder. Combat buildings expose the `IS_COMBAT`-style contract via components/tags rather than class flags.
- G-4 `game/enemies/`: raider, siege cannon, boss + wave scaling; spawn queue driven by balancing data.
- G-5 `game/core/`: phase machine (BUILDING → ENEMY → ROUND_END → [LEVELUP] → INCOME), income/payday ordering (stat snapshot → income → upkeep → painters → revive → cleanup), XP/levelup roll+apply, lives-mode base logic, combat speed control — all per prototype behavior.
- G-6 `game/ui/`: HUD, building UI, menus (main/pause/settings/credits), levelup window, game over, boss cutscene, game log — rendered through the engine overlay pass where world-anchored, direct HUD layer otherwise.
- G-7 All tunables read from `data/balancing/` at startup; no gameplay constant lives in code.
- G-8 Headless boot: game constructs and loads all data with SDL dummy drivers (the smoke test, T-2).

---

7 Editor (`editor/`) — requirements `ED-*`

7.1 Shell

- ED-1 PySide6 application, `py editor/main.py`. Docked panels: selector (left), viewport (center), balancing (bottom), asset import (right/contextual), toolbar (run controls, spawnclaude). Layout persisted to `.editor_prefs.json` (gitignored).
- ED-2 Viewport hosts the engine's offscreen surface at interactive frame rate (see phase-3 spike; fallback QImage copy accepted if □ 60 fps).
- ED-3 Selection model: exactly one selected node at a time drives viewport mode, balancing panel content, and asset-import context.

7.2 Selector panel

- ED-10 Tree: Maps (each map + "active" marker) · Buildings (type → tier → level) · Enemies · UI · VFX · Deco — populated from the slot registry + data files, never hardcoded.
- ED-11 □ marker on nodes with an assigned asset (parity with the prototype importer's tree markers).

7.3 Viewport panel

- ED-20 Map selected → tilemap editor (Godot-style):
  - Palette dock: the semantic tile types (Buildable, Combat, Spawning, background borders) each shown with its assigned sprite (grey X if none); click to arm the brush. A second palette section lists deco slots.
  - Tools: paint, erase, line, rectangle fill, bucket fill, picker (eyedropper grabs the tile type under the cursor). Ghost preview of the armed tile/deco under the cursor, snapped to the grid; click-to-paint via `screen_to_world`.
  - Layers with visibility (eye) toggles: terrain, zone tint overlay, base object, deco. The deco layer paints deco sprites that render above entities in game (E-26).
  - Base is the single movable map object — drag to reposition. Spawning is a painted tile zone; there are no spawn-point objects (D-20).
  - No autotiling. Checkerboard `_b` variant alternation is a simple position-based rule, not terrain matching.
  - Create/duplicate/save maps; set active map (writes D-21). One map open at a time.
- ED-23 Viewport navigation: right-click-drag pans, scroll-wheel zooms centered on the cursor, grid-lines toggle, world-coordinate readout in a status bar. Same feel in entity preview.
- ED-24 Global undo/redo: every editor action — paint stroke (coalesced per stroke), base move, deco place/remove, balancing edit, asset assignment — goes through ONE `QUndoStack`; Ctrl+Z / Ctrl+Y everywhere, unlimited in-session.
- ED-21 Entity selected → entity preview: the entity rendered by the real engine pipeline, idle by default, animation dropdown to preview any authored animation, range-radius overlay from its balancing values, grey X when no asset.
- ED-22 Viewport never uses a second render path — everything the editor shows goes through `engine/render` (pillar 2; what the editor shows is what the game draws).

7.4 Balancing panel

- ED-30 Form auto-generated from the selected object's schema: numeric spin/slider (with units + combat-scale hints from D-12), enums as dropdowns, booleans as checkboxes; invalid input can't be

committed.

- ED-31 Writes go through the validating writer to `data/balancing/`; every commit is undoable via the global undo stack (ED-24).
- ED-32 Locked domain → panel read-only with lock owner displayed.

7.5 Asset import

- ED-40 Full feature parity with `tools/asset_importer.py`: import sheet PNG (grid check + off-grid warning), rows = animations with row 0 idle enforced, per-row animation name / fps / hidden-frame toggles / loop range×count, entry-level offset X/Y, live animated preview honoring `playback_order`, save to manifest v2, clear-to-placeholder (removes entry + imported PNG after confirm).
- ED-41 Works for every slot category (buildings, enemies, tiles, UI, VFX), using each category's frame size and animation vocabulary.
- ED-42 After save, the entity preview (ED-21) reflects the new asset without an editor restart.

7.6 Run controls

- ED-50 Play: save all dirty data → validate → launch `py game/main.py` as a subprocess. Editor stays open; subprocess output captured to an editor console pane.
- ED-51 Build: run the PyInstaller build (tools/build script); progress + errors surfaced in the console pane.
- ED-52 Playbuild: launch `dist/HowToBeHuman/HowToBeHuman.exe`; disabled with a hint when no build exists.

7.7 Spawnclaude & access

- ED-60 Spawnclaude dialog: choose a domain (locks that domain via the protocol, opens a terminal running `claude` in the repo with the domain context injected) or small-tweak mode (no lock, explicitly scoped prompt).
- ED-61 Domains already locked are greyed out with owner shown; the editor never force-unlocks (merge-domain remains the only unlock).
- ED-62 The editor refuses to write into a domain locked by someone else (same rule agents follow) — one enforcement point for humans and agents.
- ED-63 *Open:* further access-limitation ideas pending; must not require restructuring (locks are the single enforcement point by design).

---

8 Workflow, tooling & verification — requirements `T-*`

- T-1 Branch + lock protocol ported from the prototype (`/start-domain`, `/resume-domain`, `/finish-domain`, `/merge-domain` in `.claude/commands/`), operating on `data/balancing/*.json` `_lock` fields. Invariant: while a `feature<Domain>` branch exists, that domain stays LOCKED. No destructive git on uncommitted work.
- T-2 Headless smoke test (`tools/smoke.py`): SDL dummy drivers → validate all data files against schemas → construct the game → report OK. Run after every Python/JSON change; CI-runnable.

- T-3 Unit tests where logic is pure: coords round-trip, playback_order, spatial grid queries, phase machine transitions, schema validation. Per the project's TDD workflow, each feature starts from an acceptance checklist + runnable test.
- T-4 Build script (`tools/build.*`): PyInstaller one-folder build to `dist/`; bundles `data/`; never committed.
- T-5 Every PR states a concrete in-game Quick Test scenario (not just a checklist), per project convention.

---

9 Open questions

1. Exact frame sizes per new slot category (UI, VFX, enemies) — decide when the slot registry is authored (phase 5); buildings 64×96 and tiles 64×32 carry over.
2. Access-limitation ideas beyond locks (ED-63) — pending designer input.
3. Editor console pane scope (Play output only vs. also agent session logs) — decide at phase 7/8.
4. Whether `ui`/`vfx` get their own balancing domains + locks or fold into `core` — decide when porting begins (phase 9).

Danke — let's make a banger.

Agent legibility · Strict layering · The editor is the interface

DRUNKEN DONUTS